

Container Level 0

Build It Here. Run It There.

The whole idea in one sentence

A container is a recipe plus proof that the recipe works somewhere else, too.

Companion slides for the full recorded walkthrough – see the Container Level 0 Canvas page for the video and commands.

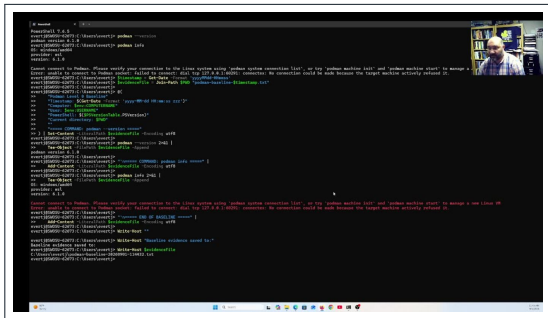
The five-step workflow

1. Describe an image with a Containerfile.
2. Build the image with Podman.
3. Run a container from the image, right here.
4. Publish the built image to GitHub Container Registry (GHCR).
5. Pull and run that same image on *another* machine.

This is Level 0 on purpose

No web framework, no database, no package manager, no build script – just enough to see the whole loop once, end to end.

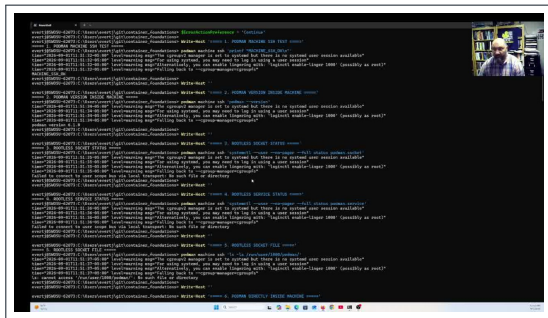
Before you build: check the machine is ready



What to notice

The recording checks Podman is actually reachable *before* trying to build anything – `podman machine list / podman info`. If your machine answers with real output instead of a connection error, you're clear to continue.

The whole recipe is two lines

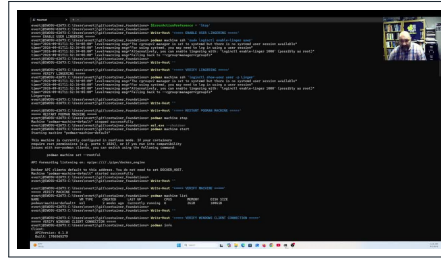


```
1 FROM docker.io/library/alpine:latest
2 CMD ["echo", "Hello, world!"]
```

The Containerfile

FROM docker.io/library/alpine:latest
CMD ["echo", "Hello, world!"]

Read it one line at a time



```
root@alpine:~# cat /etc/os-release
NAME="Alpine Linux"
ID="alpine"
VERSION=3.16.0
PRETTY_NAME="Alpine Linux 3.16.0"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://bugs.alpinelinux.org/"
root@alpine:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/sh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
tty:x:4:4:tty:/dev:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
root@alpine:~#
```

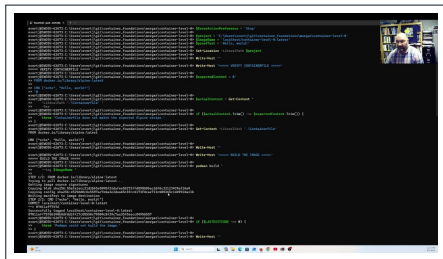
FROM

Start from an existing image (alpine:latest) instead of building an entire filesystem by hand.

CMD

The command the container runs the moment it starts: echo "Hello, world!".

Build it, then run it



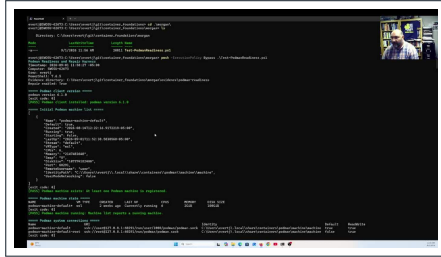
Build

`podman build -tag $imageName .` turns the Containerfile into an image and stores it locally.

Verify first

The script re-reads the Containerfile and compares it to the expected recipe *before* building – catch a typo before it costs you a build.

Run it – proof the image works

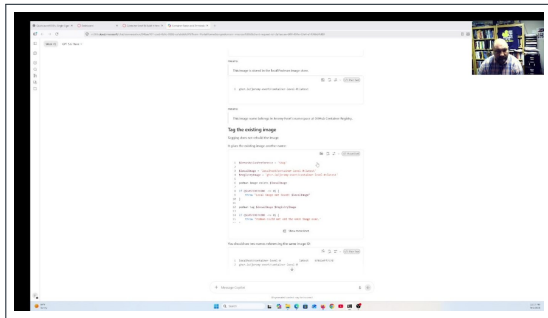


Success looks like this

```
HELLO_FROM_OUR_OWN_IMAGE
CONTAINER LEVEL 0: PASS
```

The container printed the exact text baked into the `CMD` line. That's the whole proof at this stage: build here, run here.

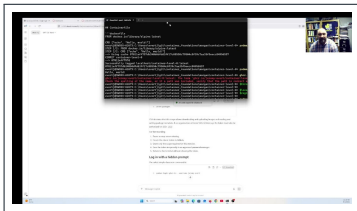
Tag it for the registry



Tagging does not rebuild the image

It gives the *same* image a second name – one your local machine understands (`localhost/container-level-0:latest`) and one GHCR understands (`ghcr.io/<your-username>/container-level-0:latest`).

Login and push – expect a little friction



This is normal, not a failure

A mistyped image path, a scope-limited token, or a first-time podman login prompt are the most common snags. Catch the exact error message and fix that one thing – do not start over from scratch.

The commands, once the path and token are right

```
gh auth token | podman login ghcr.io -u <user> --password-stdin
podman push ghcr.io/<user>/container-level-0:latest
```

Run it there – the payoff



The claim this proves

The image, not the machine, is the unit of truth. A second computer with no copy of your source code can still reproduce the exact same output.

Where to go from here

- ▶ Full recorded walkthrough and exact commands: Container Level 0 page, Computing Commons Canvas.
- ▶ If something goes wrong: see the "If something goes wrong" section on that same page – it lists the real snags from this recording (pasted-content parser errors, a stale image after an edit, a 403 on push, private-by-default visibility).
- ▶ Using Docker instead of Podman? Same commands, `docker` instead of `podman` – see the page for the exact pull command.

Next step

Try it yourself: change the CMD line, rebuild, rerun, and watch the output change. That's the whole loop, in your own hands.